

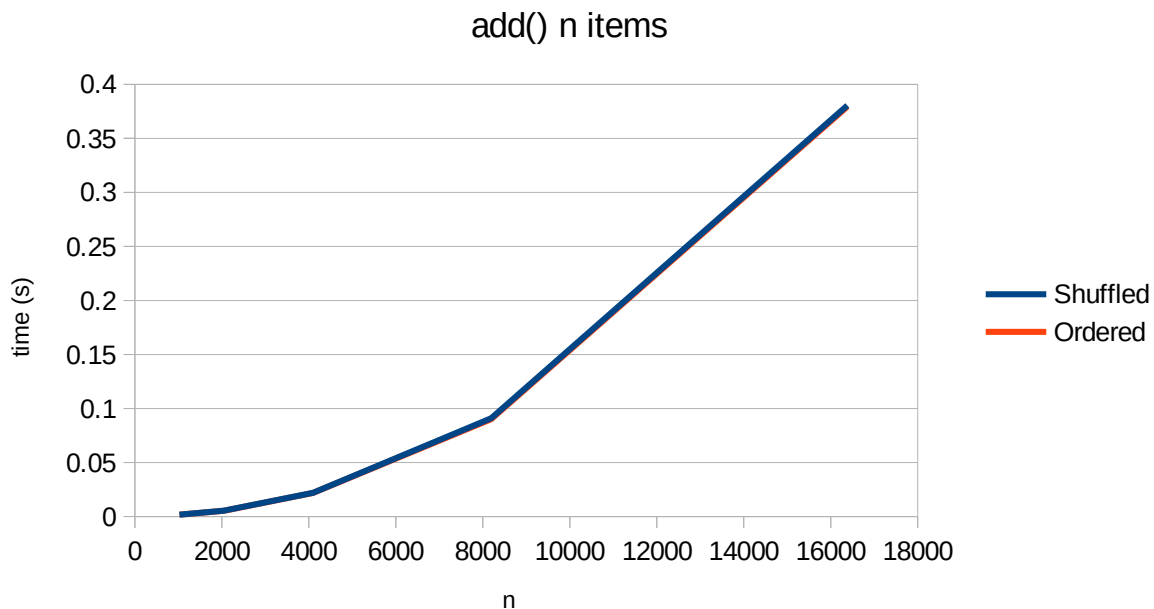
1) *Explain how the order that items are inserted into a BST affects the construction of the tree, and how this construction affects the running time of subsequent calls to the add, contains, and remove methods.*

Inserting items into a BST without considering the balance of the tree can create unbalanced trees with heights up to  $N$  (for example when inserting elements into a tree in order.) This can reduce or erase the benefits of a BST's quick add, contains, and remove – with a balanced tree they are  $O(\lg n)$ , but a fully lopsided tree will be  $O(n^2)$  because it's look-up

2) *Design and conduct an experiment to illustrate the effect of building an  $N$ -item BST by inserting the  $N$  items in sorted order versus inserting the  $N$  items in a random order. Carefully describe your experiment, so that anyone reading this document could replicate your results.*

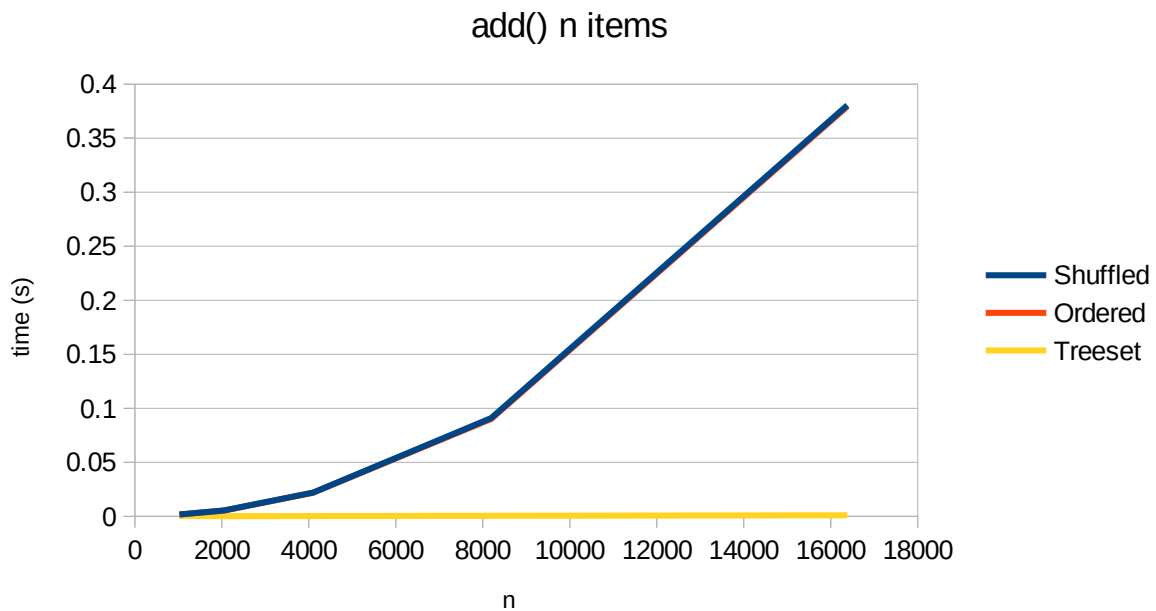
I will take int arrays of size  $2^{10}$  to  $2^{14}$  in powers of 2, ordered and shuffled, then measure how long it takes to build those arrays into BSTs.

3) *Plot the results of your experiment. Since the organization of your plot(s) is not specified here, the labels and titles of your plots(s), as well as your interpretation of the plots, are critical.*



My experiment showed nearly identical time for shuffled vs ordered. Items. This was not expected – the ordered addition should have been significantly worse.

4) *Design and conduct an experiment to illustrate the differing performance in a BST with a balance requirement and a BST that is allowed to be unbalanced. Use Java's TreeSet as an example of the former and your BinarySearchTree as an example of the latter. Carefully describe your experiment, so that anyone reading this document could replicate your results. Plot the results of your experiment. Since the organization of your plot(s) is not specified here, the labels and titles of your plots(s), as well as your interpretation of the plots, are critical.*



I think the most correct conclusions to come to when looking at this data is that there are significant flaws in my implementation of the BST, potentially that I am always currently getting worst-case performance for adding new items. It's not completely clear from the graph, but the runtime of adding to the treerset for  $10^{14}$  items was 9 milliseconds.

5) *Discuss whether a BST is a good data structure for representing a dictionary. If you think that it is, explain why. If you think that it is not, discuss other data structure(s) that you think would be better. (Keep in mind that for a typical dictionary, insertions and deletions of words are infrequent compared to word searches.)*

It has many advantages but it's lookup will never be constant, and a Hashmap offers constant lookup. That being said, if the dictionary cares about looking up similar words, which could be useful for offering spelling correction, than a BST might be the best choice.

6) *Many dictionaries are in alphabetical order. What problem will it create for a dictionary BST if it is constructed by inserting words in alphabetical order? What can you do to fix the problem?*

Naively adding dictionaries in alphabetical order will create an unbalanced tree. Shuffling the words first is a quick, simple way to, on average, create a more balanced tree.